**Zaawansowane Aplikacje Internetowe - Projekt /26Z/**

## Cel i temat projektu

Zadaniem jest zaprojektowanie, zaimplementowanie i publiczne wdrożenie aplikacji internetowej do zbierania i prezentacji wyników pomiarów pochodzących z sensorów IoT w wielu seriach. Projekt jest oceniany w skali 50 pkt (+ do 10 pkt dodatkowych) i realizowany w dwóch etapach. Ponad połowa punktów jest przyznawana pół-automatycznie: zestaw testów prowadzącego sprawdza wdrożone API i interfejs pod podanym przez Ciebie adresem i sugeruje ocenę. Testy mogą być uruchamiane samodzielnie dowolną liczbę razy.

Aplikacja prezentuje zebrane dane w formie wykresu w funkcji czasu oraz tabeli, z możliwością zawężenia prezentowanego przedziału czasu i wyboru serii widocznych na wykresie. Aplikacja służy wyłącznie do prezentacji danych: wyników pomiarów nie można dodawać, edytować ani usuwać z poziomu interfejsu. Zalogowany administrator zarządza konfiguracją, czyli seriami i czujnikami. Rodzaj mierzonej wielkości (np. temperatura, zużycie energii, wyniki treningów, kursy walut, jakość powietrza) proszę dobrać samodzielnie, zgodnie ze swoimi zainteresowaniami.

Źródłem danych są autonomiczne sensory, które przesyłają wyniki bezpośrednio do API, z pominięciem interfejsu użytkownika. Częścią rozwiązania jest obowiązkowo narzędzie (np. skrypt w Pythonie) emulujące sensor, które generuje dane lokalnie (lub np. odczytuje z publicznie dostępnych serwisów pogodowych) i przesyła je do backendu. Udział rzeczywistego sprzętu nie jest wymagany. Każdy czujnik uwierzytelnia się własnym kluczem API, który administrator otrzymuje przy rejestracji czujnika w aplikacji.

## Wymagania funkcjonalne

Wszystkie wymagania F1–F13 są obowiązkowe, a ich realizacja jest oceniana.

- **F1. Wynik pomiaru** to co najmniej jedna liczba zmiennoprzecinkowa ze znacznikiem czasu i identyfikatorem serii. Wyniki trafiają do aplikacji wyłącznie z czujników, przez API.
- **F2. Seria** grupuje wyniki z jednego eksperymentu lub z czujników przypisanych do serii. Ma nazwę, dopuszczalny przedział wartości min/max i konfigurowalną cechę graficzną (np. kolor lub ikonę). W tabeli seria to osobna kolumna, a na wykresie osobna krzywa.
- **F3. Role:** czytelnik (niezalogowany) przegląda dane; administrator (zalogowany) dodatkowo zarządza seriami (tworzenie, zmiana nazwy, przedziału i wyglądu, usuwanie) oraz czujnikami (F12). Nikt nie może dodawać, edytować ani usuwać wyników pomiarów z poziomu interfejsu.
- **F4. Walidacja zakresu:** serwer odrzuca wynik spoza przedziału min/max serii (także zmianę przedziału serii, która wykluczyłaby zapisane już wyniki), a generator danych (F13) wyświetla czytelny komunikat o odrzuceniu. Formularze administratora walidują dane także w interfejsie. Informacja o odrzuceniu wyniku pojawia się w **logu błędów**.
- **F5. Filtrowanie:** użytkownik zawęża przedział czasu i wybiera serie widoczne na wykresie.
- **F6. Powiązanie tabeli z wykresem:** kliknięcie rekordu w tabeli wyróżnia odpowiadający mu punkt na wykresie.
- **F7. Wydruk:** każdy użytkownik może wydrukować skonfigurowany wykres wraz z tabelą, bez elementów sterujących (przycisków, odnośników, formularzy).
- **F8. Konto administratora:** logowanie, wylogowanie i zmiana hasła przez zalogowanego użytkownika.
- **F9. UX:** m.in. zatwierdzanie formularzy klawiszem Enter, walidacja po stronie interfejsu przed wysłaniem, informacja o trwającym ładowaniu i błędach komunikacji z serwerem.

- **F10. Responsywność:** interfejs działa poprawnie na telefonie (szerokość od 360 px), tablecie i komputerze.
- **F11. Przykładowe dane:** aplikacja zawiera co najmniej 3 serie po co najmniej 15 punktów, pozwalające szybko zapoznać się z jej działaniem. Dane muszą zostać wprowadzone generatorem (F13) przez API, a nie bezpośrednio do bazy danych.
- **F12. Zarządzanie czujnikami:** administrator rejestruje nowy czujnik, podając jego nazwę i serię, do której czujnik zapisuje wyniki. Po rejestracji aplikacja jednorazowo wyświetla klucz API czujnika. Administrator widzi listę czujników i może wyrejestrować dowolny z nich; klucz wyrejestrowanego czujnika natychmiast przestaje działać, a zebrane przez niego wyniki pozostają.
- **F13. Generator danych (emulator czujnika):** narzędzie dostarczane jako część rozwiązania, które wysyła wyniki do API aplikacji. Użytkownik narzędzia ustawia co najmniej: adres API, klucz API czujnika, liczbę wyników lub odstęp czasu między nimi oraz sposób generowania wartości (np. zakres wartości albo przebieg losowy lub sinusoidalny). Narzędzie umożliwia zarówno wysłanie serii wyników ze znacznikami czasu z przeszłości (do przygotowania danych przykładowych), jak i wysyłanie bieżących wyników w stałych odstępach czasu.

## Wymagania techniczne

Technologie proszę wybrać samodzielnie w ramach poniższych ograniczeń; wybór i jego uzasadnienie należy opisać w dokumentacji.

- **T1. Backend.** Do wyboru: Node.js (np. Express, Fastify, NestJS), Python (np. Django, FastAPI, Flask), PHP (np. Laravel, Symfony), C# (ASP.NET Core) lub Java (Spring Boot).
- **T2. Frontend.** Aplikacja jednostronicowa (SPA) zbudowana z użyciem frameworka lub biblioteki (np. React, Vue, Angular, Svelte). Wygląd definiują style CSS; dopuszczalne są biblioteki komponentów, ale układ responsywny musi wykorzystywać Flexbox lub Grid oraz zapytania medialne.
- **T3. REST API zgodne ze wspólnym kontraktem.** Wszyscy implementują to samo API, opisane w pliku `zai-api-26z.yaml` (OpenAPI 3) udostępnionym na platformie Leia. Kontrakt jest zgodny z poziomem 2 modelu dojrzałości Richardsona i określa ścieżki, formaty danych, kody odpowiedzi, sposób logowania oraz format błędów (Problem Details, RFC 9457). Wszystkie ścieżki mają prefiks `/api`, a w skrócie wyglądają następująco:

| Zasób | Operacje | Dostęp |
|---|---|---|
| `/api/auth/login`, `/api/auth/logout`, `/api/auth/password` | logowanie, wylogowanie, zmiana hasła | administrator |
| `/api/series`, `/api/series/{id}` | lista i odczyt serii; tworzenie, modyfikacja i usuwanie serii | odczyt - publiczny; zmiany - administrator |
| `/api/sensors`, `/api/sensors/{id}` | lista czujników, rejestracja (zwraca klucz API), wyrejestrowanie | administrator |
| `/api/measurements (POST)` | przesłanie wyniku; seria wynika z klucza czujnika | czujnik (nagłówek X-API-Key) |
| `/api/measurements?series=…&from=…&to=…`, `/api/measurements/{id}` | wyniki wybranych serii w przedziale czasu, odczyt wyniku | publiczny |

API nie udostępnia modyfikacji ani usuwania wyników pomiarów; znikają one wyłącznie razem z usuwaną serią. Kontrakt wolno rozszerzać o własne pola i ścieżki, ale nie wolno zmieniać ani usuwać elementów w nim opisanych, bo od tego zależą testy automatyczne. Interfejs użytkownika, struktura bazy danych i dziedzina pomiarów pozostają elementem projektu.

- **T4. Baza danych.** Relacyjna (np. PostgreSQL, MySQL/MariaDB, SQLite). Struktura musi odpowiadać problemowi: właściwe typy pól, klucze główne i obce, ograniczenia, indeksy dla zapytań po czasie. Schemat tworzony skryptem SQL lub migracjami.
- **T5. Bezpieczeństwo.** Obowiązkowe jest pięć elementów:
    1. hasła przechowywane z użyciem funkcji przeznaczonej do haseł (Argon2id lub bcrypt),
    2. zapytania parametryzowane lub ORM, bez sklejania SQL z danymi użytkownika,
    3. bezpieczny mechanizm sesji lub tokenów (np. ciasteczka HttpOnly, Secure, SameSite; tokeny z ograniczonym czasem ważności),
    4. autoryzacja wszystkich operacji modyfikujących na serwerze, nie tylko ukrywanie przycisków w interfejsie,
    5. uwierzytelnianie czujników losowymi kluczami API o długości co najmniej 32 znaków, przechowywanymi w bazie wyłącznie jako skrót (np. SHA-256) i nigdy nieujawnianymi po rejestracji.
- **T6. Dostępność.** Interfejs spełnia kluczowe kryteria WCAG 2.2 na poziomie AA: semantyczny HTML, etykiety pól formularzy, obsługa z klawiatury, widoczny fokus, kontrast kolorów, teksty alternatywne. Seria na wykresie nie może być rozróżniana wyłącznie kolorem. Część tych kryteriów sprawdza automatycznie audyt Lighthouse (A5); przed zgłoszeniem proszę uruchomić go samodzielnie w narzędziach deweloperskich przeglądarki.
- **T7. Wydruk.** Realizowany jako ten sam widok co w aplikacji, modyfikowany wyłącznie regułami `@media print` ukrywającymi kontrolki.
- **T8. Testy.** Udostępniony został Państwu zestaw testów automatycznych w postaci skryptu, który sprawdza wdrożoną aplikację pod podanym adresem. Można go uruchamiać dowolną liczbę razy. Wynik testów w dniu terminu jest podstawą oceny części automatycznej. Własne testy lub zgłoszenia błędów w skrypcie testującym są mile widziane i mogą dać punkty dodatkowe (sekcja 5).
- **T9. Repozytorium.** Kod proszę rozwijać w repozytorium Git z czytelną historią zmian, obejmującą cały okres realizacji projektu. Link do repozytorium (publicznego lub udostępnionego prowadzącemu) proszę podać już w etapie E1. Historia commitów jest brana pod uwagę przy weryfikacji wyrywkowej.
- **T10. Wdrożenie.** Aplikację należy udostępniać pod publicznym adresem do czasu wystawienia oceny. Do tego celu można użyć np. własnego serwera, studenckiego konta w chmurze lub platformy PaaS z darmowym planem. Uwaga: typowe darmowe hostingi WWW obsługują tylko PHP.
- **T11. Generator danych.** Technologię i interfejs narzędzia z F13 można wybrać dowolnie: może to być program konsolowy, osobna aplikacja webowa albo odrębny widok w aplikacji prezentującej wyniki. Generator zawsze przesyła wyniki przez API aplikacji (`POST /api/measurements` z kluczem czujnika); bezpośredni zapis do bazy danych jest niedopuszczalny. Kod generatora znajduje się w tym samym repozytorium co aplikacja, a dokumentacja opisuje sposób jego uruchomienia i dostępne parametry. Klucz API nie może być zapisany na stałe w kodzie generatora; podaje się go jako parametr, zmienną środowiskową lub w polu formularza.

## Etapy i terminy

Projekt należy oddać w dwóch etapach, wypełniając formularz zgłoszenia na platformie Leia do godz. 23:59 wskazanego dnia. Po każdym terminie prowadzący uruchamia testy automatyczne dla wszystkich zgłoszonych adresów. Wyniki punktowe zostaną podane w systemie Leia.

| Etap | Termin | Co musi działać | Co podlega zgłoszeniu |
|---|---|---|---|
| **E1. Wdrożone API** | 7 lis 2026 | Backend zgodny z kontraktem (w tym obsługa czujników), z bazą danych, wdrożony publicznie; generator danych i dane przykładowe (F11) wprowadzone przez API | Adres API, dane testowego konta administratora, link do repozytorium |
| **E2. Aplikacja końcowa** | 21 lis 2026 | Pełna aplikacja z interfejsem, wdrożona publicznie | Adres aplikacji, nagranie demonstracyjne, lista kontrolna, archiwum ZIP z kodem i dokumentacją |

W etapie E2 testy API są uruchamiane ponownie, więc błędy z etapu E1 warto poprawić. Etap E1 niezgłoszony w terminie może zostać zgłoszony razem z E2 za maksymalnie 5 pkt.

Testowe konto administratora podane w formularzu musi działać do czasu wystawienia oceny. Testy tworzą własne serie, czujniki i wyniki, a na końcu usuwają utworzone serie (razem z ich czujnikami i wynikami); nie zmieniają zawartych w bazie danych przykładowych Studenta.

## Punktacja

Maksymalnie 50 pkt oraz do 10 pkt dodatkowych, przy czym suma punktów projektu nie przekracza 50.

**Część A (26 pkt)** przyznawana jest przez prowadzącego na podstawie wyników testów automatycznych (dopuszczalne są korekty w przypadku zaobserwowania nieprawidłowości w działaniu skryptu):

| Kod | Etap | Co sprawdzają testy | Pkt |
|---|---|---|---|
| **A1** | E1 | Zgodność API z kontraktem: ścieżki, metody, kody odpowiedzi, nagłówek Location, schematy danych, rejestracja i wyrejestrowanie czujników, przyjmowanie wyników z kluczem API, filtrowanie po czasie i seriach, obecność danych przykładowych (F11) | 10 |
| **A2** | E2 | Ponownie zgodność API z kontraktem | 6 |
| **A3** | E2 | Walidacja i autoryzacja: odrzucenie wartości spoza min/max (422), błędnych danych (400/422), operacji administracyjnych bez logowania oraz wyników bez ważnego klucza czujnika (401); brak możliwości modyfikacji i usuwania wyników | 4 |
| **A4** | E2 | Konfiguracja bezpieczeństwa: HTTPS, nagłówki bezpieczeństwa z CSP, flagi ciasteczek sesji, brak haseł, ich skrótów i kluczy API czujników w odpowiedziach API | 3 |
| **A5** | E2 | Dostępność: wynik Lighthouse (kategoria Accessibility) dla strony głównej; ≥ 90 = 3 pkt, 80–89 = 2 pkt, 70–79 = 1 pkt | 3 |

Punkty w A1–A4 są proporcjonalne do odsetka zaliczonych testów w danej grupie. Testy są uruchamiane dwukrotnie w odstępie kilku godzin; liczy się lepszy wynik, co chroni przed chwilową niedostępnością darmowego hostingu.

**Część B (24 pkt)** przyznawana jest na podstawie nagrania i listy kontrolnej, w których każdy punkt jest oceniany zero-jedynkowo:

| Kod | Co musi być widoczne | Pkt |
|---|---|---|
| **B1** | Wymagania F1–F13 pokazane w nagraniu, po 1 pkt za każde | 13 |
| **B2** | Widok mobilny (szerokość ok. 360 px) oraz podgląd wydruku bez kontrolek | 4 |
| **B3** | Dwa wybrane elementy bezpieczeństwa z T5, pokazane w nagraniu i opisane w dokumentacji | 3 |
| **B4** | Diagram ERD i skrypt lub migracje bazy danych w archiwum | 2 |
| **B5** | Kompletna dokumentacja | 2 |

Punkty dodatkowe (do 10 pkt łącznie):

| Rozszerzenie | Jak dostarczone | Pkt |
|---|---|---|
| Aktualizacja wykresu w czasie rzeczywistym (WebSocket lub SSE), gdy generator wysyła bieżące wyniki | nagranie | do 3 |
| Generator pobierający rzeczywiste dane z publicznego źródła (np. serwisu pogodowego lub jakości powietrza) | nagranie i kod | do 2 |
| Własne testy automatyczne uruchamiane w CI (np. GitHub Actions), z zielonym statusem | link w formularzu | do 2 |
| Użycie bazy danych szeregów czasowych (np. InfluxDB, TimescaleDB, MongoDB, itp.) w miejsce klasycznej bazy relacyjnej | nagranie i kod | do 3 |

**Weryfikacja wyrywkowa.** Około 15% projektów, wybranych losowo oraz spośród zgłoszeń z nietypowymi wynikami, jest sprawdzanych szczegółowo: prowadzący uruchamia aplikację i przegląda kod oraz historię repozytorium. Jeśli nagranie lub lista kontrolna nie odpowiadają rzeczywistemu działaniu aplikacji, punkty za dany element nie są przyznawane, a od oceny odejmuje się dodatkowo 5 pkt.

### Zawartość zgłoszenia końcowego (E2)

W etapie E2 przesyłasz na platformę Leia formularz zgłoszenia, nagranie demonstracyjne, listę kontrolną oraz archiwum ZIP z kodem i dokumentacją.

**Formularz zgłoszenia** zawiera: adres aplikacji, adres API (jeśli inny), dane testowego konta administratora, link do repozytorium oraz opcjonalnie link do statusu CI.

**Nagranie demonstracyjne** trwa od 5 do 7 minut i pokazuje działającą, wdrożoną aplikację (nie wersję lokalną) z widocznym paskiem adresu. Kolejno prezentowane są wymagania F1–F13, widok mobilny, podgląd wydruku, dwa wybrane elementy bezpieczeństwa oraz ewentualne rozszerzenia. Nagranie musi obejmować pełny cykl pracy z czujnikiem: rejestrację czujnika w aplikacji, ustawienie jego klucza w generatorze, wysłanie wyników i ich pojawienie się na wykresie, odrzucenie wyniku spoza przedziału oraz odmowę przyjęcia wyników po wyrejestrowaniu czujnika. Przed każdym wymaganiem proszę wyświetlić lub powiedzieć jego kod (np. „F6”). Nagranie proszę przesłać jako plik MP4 (do 100 MB).

**Lista kontrolna** to formularz, w którym dla każdego elementu B1–B5 i rozszerzeń deklarujesz realizację oraz podajesz minutę nagrania, w której jest widoczny.

**Archiwum ZIP** (bez katalogów zależności, np. `node_modules`) zawiera kod źródłowy aplikacji i generatora danych, skrypt SQL lub migracje tworzące strukturę bazy danych (dane przykładowe wprowadza generator), plik `.env.example` bez prawdziwych haseł oraz dokumentację PDF.

**Dokumentacja PDF** (do 8 stron) powinna zawierać:

1. Adres aplikacji wdrożonej na publicznie dostępny serwerze oraz link do repozytorium.
2. Dane logowania dla konta administratora.
3. Instrukcję lokalnego uruchomienia aplikacji (wersje narzędzi, polecenia, dane logowania) oraz generatora danych, z opisem jego parametrów.
4. Diagram ERD z krótkim opisem tabel i relacji.
5. Opis dwóch wybranych elementów bezpieczeństwa (T5): co, gdzie w kodzie i przed czym chroni.
6. Opis ewentualnych rozszerzeń kontraktu API.
7. Deklarację użycia narzędzi AI (patrz: zasady realizacji).

## Zasady realizacji

**Samodzielność.** Projekt należy realizować indywidualnie. Można korzystać z dokumentacji, tutoriali i bibliotek, podając źródła fragmentów kodu przejętych z zewnątrz. Kod wszystkich projektów jest automatycznie porównywany pod kątem podobieństwa. W przypadku projektów objętych weryfikacją wyrywkową lub o wysokim podobieństwie kodu prowadzący może poprosić o krótką rozmowę (do 15 minut, zdalnie) w celu wyjaśnienia wybranych fragmentów kodu i decyzji projektowych.

**Narzędzia AI.** Korzystanie z asystentów AI (np. do generowania fragmentów kodu, testów lub dokumentacji) jest dozwolone. W dokumentacji należy zamieścić krótką deklarację: jakich narzędzi użyto i do czego. Odpowiadasz za cały oddany kod, w tym za jego poprawność i bezpieczeństwo.

**Terminy.** Testy automatyczne są uruchamiane po terminie właściwym dla danego etapu, więc zmiany we wdrożonej aplikacji mogą wpłynąć na wynik. Sugeruję powstrzymać się od wdrażania nowych wersji do czasu publikacji ocen z etapu.

**Dostępność wdrożenia.** Proszę nie wyłączać aplikacji ani nie zmieniać adresu do czasu wystawienia oceny. Jeśli używany jest darmowy plan, który usypia aplikację po bezczynności, proszę to wyraźnie zaznaczyć w dokumentacji oraz podać przybliżony czas pierwszego uruchomienia.

**Pomoc.** Pytania dotyczące projektu proszę zamieszczać na forum kursu.